Advanced Serverless Architecture: VPC Integration with AWS Lambda and Amazon RDS Proxy

AWS DVA-C02 Preparation Project Report

July 2026

Abstract

This report details the architectural design, deployment, and validation of an advanced serverless database integration on Amazon Web Services (AWS). The project addresses the prevalent issue of connection exhaustion when highly scalable compute layers (AWS Lambda) interface with traditional relational database management systems (Amazon RDS). By implementing Amazon RDS Proxy, strict VPC network isolation, and dynamic identity management via AWS Secrets Manager, this project demonstrates best practices aligned with the AWS Certified Developer - Associate (DVA-C02) examination standards.

1 Introduction

Modern serverless applications utilize AWS Lambda to achieve horizontal scalability and cost optimization. However, a fundamental architectural mismatch occurs when Lambda functions must interact with relational databases.

1.1 The Connection Exhaustion Problem

AWS Lambda scales horizontally by spinning up isolated execution environments for concurrent requests. If an application experiences a sudden surge of 1,000 concurrent requests, AWS Lambda provisions 1,000 independent environments. If connected directly to a relational database, this results in 1,000 simultaneous connection attempts. Relational databases allocate substantial memory per connection and will rapidly exhaust their resources, leading to connection timeouts, dropped requests, and system failure.

1.2 Project Objectives

This project mitigates the connection exhaustion bottleneck through the following objectives:

- Deploy an Amazon RDS PostgreSQL database within a highly available, multi-AZ Virtual Private Cloud (VPC) environment.
- Implement Amazon RDS Proxy to establish a connection pool, multiplexing thousands of Lambda connections down to a sustainable footprint on the database.
- Enforce a rigid ”Chain of Trust” using Security Groups to ensure zero external ingress to the database layer.
- Utilize Infrastructure as Code (IaC) via AWS CloudFormation to ensure deterministic, repeatable deployments.

2 Architecture and Security Implementation

2.1 Network Topology and Subnetting

The infrastructure is deployed within a custom VPC configured with both Public and Private subnets spanning across two Availability Zones (AZs) to satisfy AWS High Availability (HA) constraints.

- **Private Subnets:** House the AWS Lambda functions, the RDS Proxy endpoints, and the Amazon RDS database instance. These subnets are explicitly associated with a private route table.
- **Public Subnets:** House the NAT Gateway, routing external API calls from the private subnets to the Internet Gateway (IGW).

2.2 Security Groups and Chain of Trust

Network access is restricted using a linear progression of Security Groups (SGs). Direct database access is blocked entirely.

1. **Lambda SG:** Permits outbound traffic (Egress '0.0.0.0/0') allowing Lambda to communicate with the proxy and external endpoints via the NAT Gateway.
2. **RDS Proxy SG:** Permits inbound TCP traffic on port 5432 *exclusively* from the Lambda SG.
3. **Database SG:** Permits inbound TCP traffic on port 5432 *exclusively* from the RDS Proxy SG.

2.3 Dynamic Identity and Secrets Management

Hardcoded credentials present a critical security vulnerability. This architecture utilizes **AWS Secrets Manager** to dynamically generate a 16-character master database password during CloudFormation stack creation. The RDS Proxy assumes an IAM role with `secretsmanager:GetSecretValue` permissions to securely fetch credentials, completely bypassing the need for Lambda to handle or store database passwords.

3 Developer Operations & Performance Tuning

3.1 Mitigating VPC Cold Starts

Historically, attaching AWS Lambda to a VPC incurred significant cold start penalties due to Elastic Network Interface (ENI) provisioning. This project leverages the modern AWS Hyperplane ENI architecture, which shares network interfaces across execution environments. To further reduce the initialization latency of the Node.js runtime and the PostgreSQL driver, the Lambda memory was tuned to **1024 MB**, which proportionally increases the allocated vCPU compute power.

3.2 Execution Context Reuse

In accordance with DVA-C02 best practices, the database connection pool logic within the Node.js Lambda code is initialized *outside* the main execution handler.

```
// Initialized outside the handler for Execution Context Reuse
let pool;
exports.handler = async (event, context) => {
    if (!pool) {
        pool = new Pool({ ... });
    }
    // ... query execution ...
};
```

This design ensures that when AWS Lambda reuses a warm container for subsequent invocations, the application does not pay the penalty of re-establishing a TCP handshake and SSL negotiation with the RDS Proxy.

4 Validation and Load Testing

To validate the multiplexing capabilities of the RDS Proxy, a controlled batching script was utilized. Due to default AWS account concurrency limits (typically 10-1000 depending on account age), the load test was orchestrated via the AWS CLI to fire 100 batches of 10 concurrent requests (1,000 total invocations).

4.1 Performance Metrics

The validation results were observed utilizing Amazon CloudWatch and PostgreSQL internal schema queries (`pg_stat_activity`).

Metric	**Observation During Load Test**
Lambda Invocations	1,000 requests processed successfully.
Lambda Throttling Errors	0 (managed via controlled CLI batching).
Client Connections (to Proxy)	Surged proportionately to Lambda scaling.
Database Connections (Backend)	Remained stable at < 5 active connections.

The load test conclusively demonstrated that the RDS proxy successfully intercepted the rapid influx of Lambda connections, utilized its internal pool to process the queries, and protected the micro-instance database from resource exhaustion.

5 Conclusion

This project successfully demonstrates a highly resilient, scalable, and secure interaction pattern between serverless compute and relational data stores. By utilizing CloudFormation for repeatable deployments, Secrets Manager for dynamic credential handling, and RDS Proxy for connection multiplexing, the architecture fulfills enterprise-grade requirements. The implementations outlined in this report serve as strong practical evidence of the competencies required for the AWS DVA-C02 certification.